マスタ編集ダイアログUI設計書

メタデータ

title: マスタ編集ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [data-admin, master, dialog, edit, creator, vendor, instrument]

関連ドキュメント

- マスタ管理メイン画面
- マスタ追加ダイアログ
- マスタ管理API仕様
- CreatorテーブルDDL
- VendorテーブルDDL
- InstrumentテーブルDDL
- バックエンド実装計画
- UIポリシー

---

1. 画面レイアウト

1.1. 画面イメージ



2. 概要

2.1. 目的

データ管理者が既存のマスタデータ（Creator、Vendor、Instrument）を編集するためのダイアログを定義します。マスタ種別に応じた入力項目を表示し、名前とアクティブ状態を変更できます。マスタIDは変更不可（主キーのため）。

2.2. 利用権限

- **Data Admin（データ管理者）**: 使用可能

---

3. ダイアログ構成

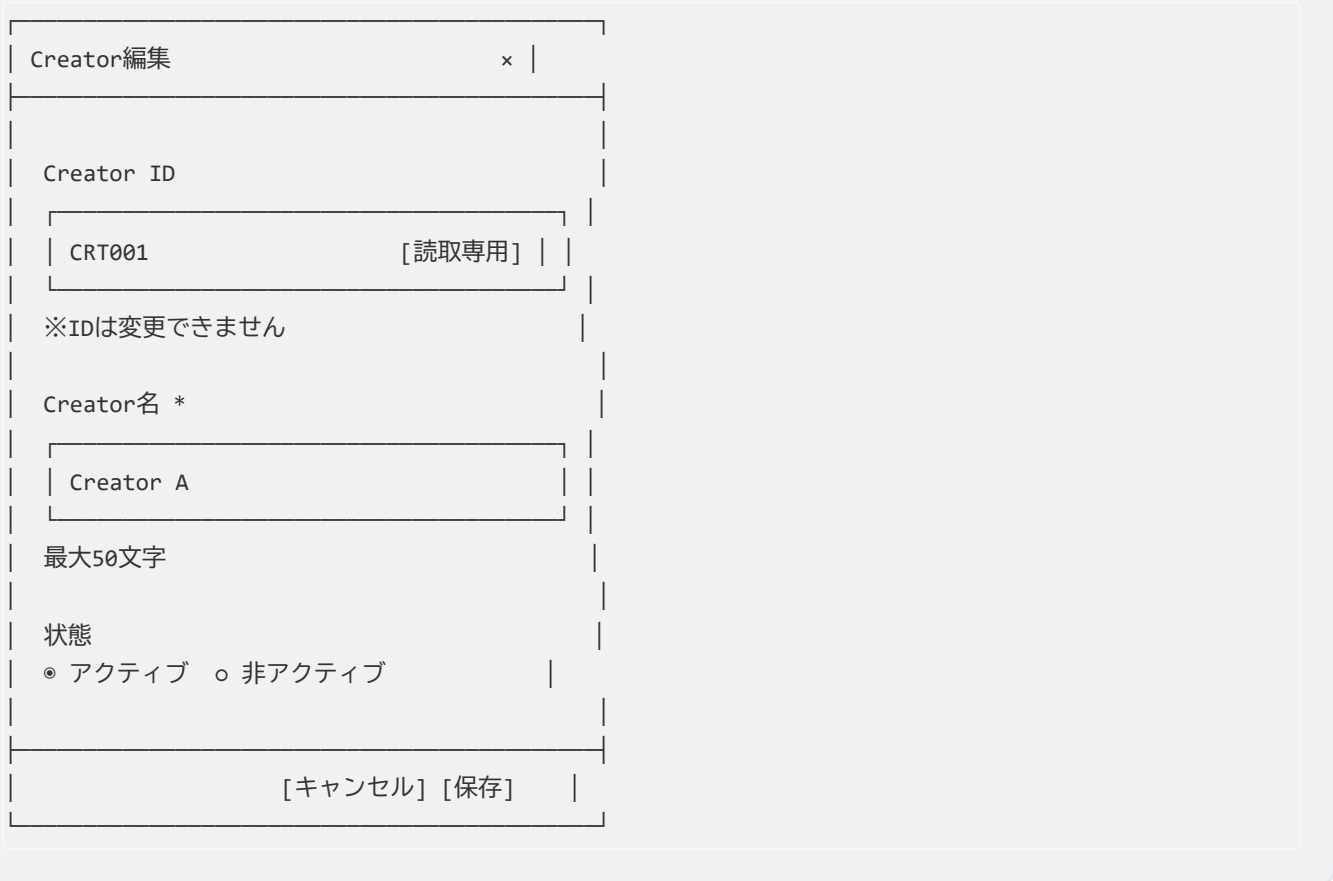
3.1. ダイアログ概要

表示トリガー: マスタ管理画面の「編集」ボタンクリック

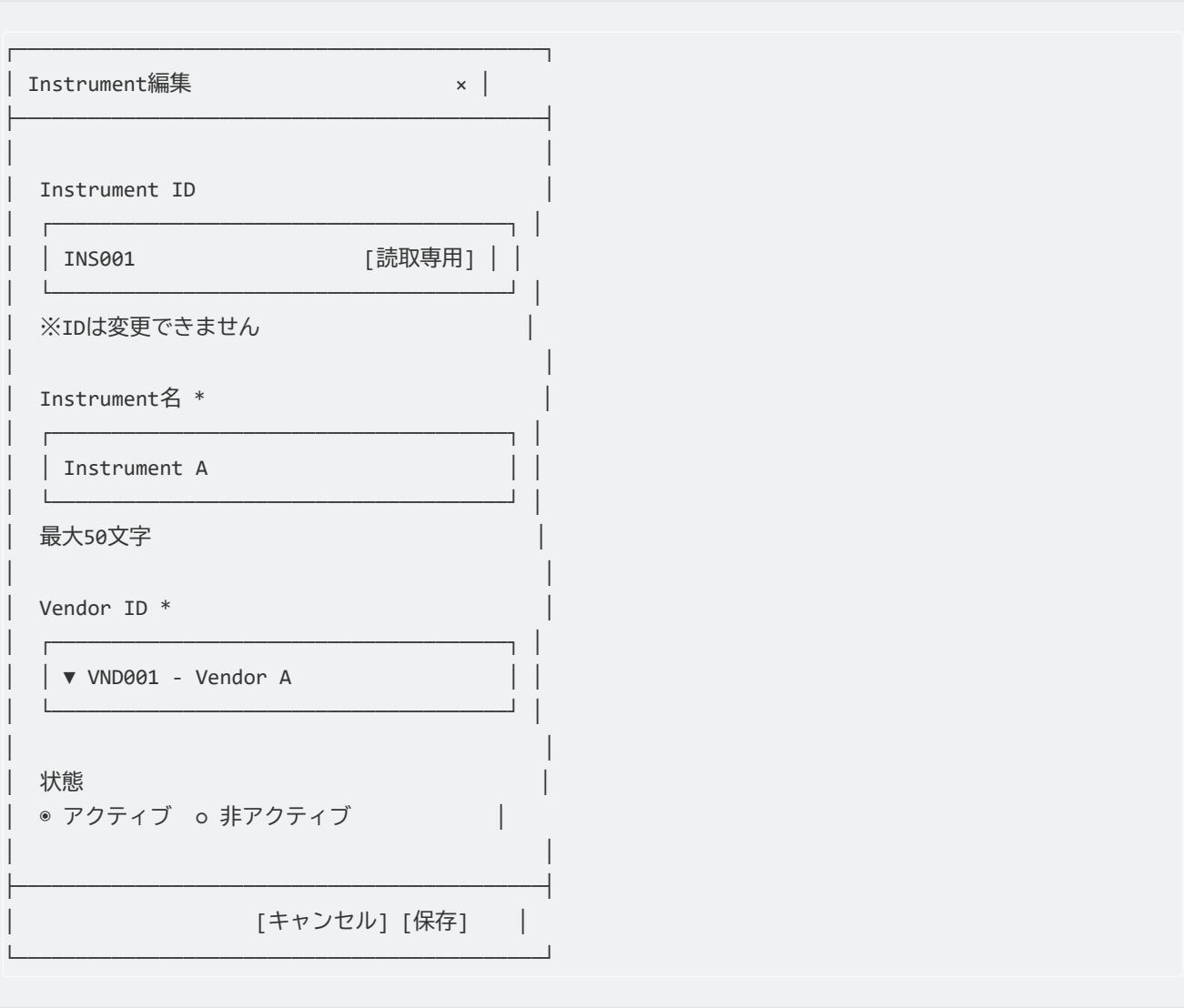
モーダルタイプ: Modal

サイズ: 中（600px）

3.2. レイアウト（Creator / Vendor の場合）



3.3. レイアウト（Instrument の場合）



---

4. 入力項目

4.1. マスタID（表示のみ）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	マスタ種別に応じて変動（「Creator ID」/「Vendor ID」/「Instrument ID」）
型	text

読み取り専用	○（disabled）
説明	「※IDは変更できません」

実装例:

```
<Form.Item
  label={labels.id}
  name="id"
  extra="※IDは変更できません"
>
  <Input disabled />
</Form.Item>
```

4.2. マスタ名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	マスタ種別に応じて変動（「Creator名」/「Vendor名」/「Instrument名」）
プレースホルダー	「名前を入力」
型	text
必須	○
最小文字数	1文字
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '名前を入力してください'
},
{
  min: 1,
  max: 50,
```

```
  message: '名前は1〜50文字で入力してください'
}
```

4.3. Vendor ID（Instrumentのみ必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「Vendor ID」
プレースホルダー	「Vendor IDを選択」
必須	○（Instrumentの場合のみ）
選択肢	アクティブなVendorマスタから取得
検索	有効（showSearch）
幅	100%

バリデーション:

```
{
  required: true,
  message: 'Vendor IDを選択してください'
}
```

4.4. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ
必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

---

5. アクションボタン

5.1. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`
配置	フッター左側

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

5.2. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
配置	フッター右側
ローディング	API呼び出し中は `true`
無効化	バリデーションエラー時

動作:

1. フォームバリデーション実行
2. 変更があるか確認
3. API呼び出し: `PUT /api/v1/master/{masterType}/:id`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

6. データの初期化

6.1. ダイアログ表示時

編集対象のマスタデータを取得してフォームに設定:

```
useEffect(() => {
  if (open && masterId) {
    fetchMasterData(masterId);
  }
}, [open, masterId]);

const fetchMasterData = async (id: string) => {
  try {
    const response = await apiClient.get(
      `/api/v1/master/${masterType}/${id}`,
```

```
      {
        headers: {
          'x-organization-id': organizationId
        }
      }
    );

    if (response.data.success) {
      const data = response.data.data;
      const formValues: any = {
        id: data[`${masterType}Id`],
        name: data[`${masterType}Name`],
        isActive: data.isActive
      };

      // Instrumentの場合のみvendorIdを設定
      if (masterType === 'instrument') {
        formValues.vendorId = data.vendorId;
      }

      form.setFieldsValue(formValues);
    }
  } catch (error) {
    message.error('マスタ情報の取得に失敗しました');
  }
};
```

---

7. APIリクエスト仕様

7.1. データ取得エンドポイント

`GET /api/v1/master/{masterType}/:id`

リクエストヘッダー:

```
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス（成功時 - Creator）:

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "creatorId": "CRT001",
    "creatorName": "Creator A",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

レスポンス（成功時 - Vendor）：

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "vendorId": "VND001",
    "vendorName": "Vendor A",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": { "..." }
}
```

レスポンス（成功時 - Instrument）：

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "instrumentId": "INS001",
    "instrumentName": "Instrument A",
    "vendorId": "VND001",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
```

```
  },
  "meta": { "..." }
}
```

7.2. 更新エンドポイント

`PUT /api/v1/master/{masterType}/:id`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

リクエストボディ（Creator）:

```
{
  "creatorName": "Creator A Updated",
  "isActive": true
}
```

リクエストボディ（Vendor）:

```
{
  "vendorName": "Vendor A Updated",
  "isActive": true
}
```

リクエストボディ（Instrument）:

```
{
  "instrumentName": "Instrument A Updated",
  "vendorId": "VND002",
  "isActive": true
}
```

レスポンス（成功時 - Creator）:

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "creatorId": "CRT001",
    "creatorName": "Creator A Updated",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:35:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

レスポンス（エラー時）:

```
{
  "success": false,
  "error": {
    "code": "MASTER_NOT_FOUND",
    "message": "マスタが見つかりません",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

---

8. エラーメッセージ

8.1. バリデーションエラー

エラー条件	メッセージ

名前が空	「名前を入力してください」
名前が長すぎる	「名前は1〜50文字で入力してください」
Vendor IDが未選択（Instrument）	「Vendor IDを選択してください」
状態が未選択	「状態を選択してください」

8.2. APIエラー

エラーコード	メッセージ
`MASTER_NOT_FOUND`	「マスタが見つかりません」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`VENDOR_NOT_FOUND`	「指定されたVendor IDが見つかりません」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

---

9. 実装例（TypeScript + Next.js + Ant Design）

```tsx
// frontend/src/components/DataAdmin/MasterEditDialog.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Select, Radio, Button, message } from 'antd';
import apiClient from '@/utils/apiClient';

type MasterType = 'creator' | 'vendor' | 'instrument';

interface MasterEditDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
  masterType: MasterType;
  masterId: string | null;
  organizationId: string;
}

interface MasterFormValues {
  id: string;
  name: string;
```

```
  vendorId?: string;
  isActive: boolean;
}

const MASTER_LABELS = {
  creator: { id: 'Creator ID', name: 'Creator名', title: 'Creator編集' },
  vendor: { id: 'Vendor ID', name: 'Vendor名', title: 'Vendor編集' },
  instrument: { id: 'Instrument ID', name: 'Instrument名', title: 'Instrument編集' }
};

export default function MasterEditDialog({
  open,
  onClose,
  onSuccess,
  masterType,
  masterId,
  organizationId
}: MasterEditDialogProps) {
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isFetching, setIsFetching] = useState(false);
  const [vendorOptions, setVendorOptions] = useState<any[]>([]);

  const labels = MASTER_LABELS[masterType];

  // Vendor一覧取得（Instrumentの場合）
  useEffect(() => {
    if (open && masterType === 'instrument') {
      fetchVendors();
    }
  }, [open, masterType]);

  const fetchVendors = async () => {
    try {
      const response = await apiClient.get('/api/v1/master/vendor', {
        params: { is_active: true },
        headers: { 'x-organization-id': organizationId }
      });

      const options = response.data.data.map((vendor: any) => ({
        label: `${vendor.vendorId} - ${vendor.vendorName}`,
        value: vendor.vendorId
      }));

      setVendorOptions(options);
    } catch (error) {
      message.error('Vendor一覧の取得に失敗しました');
    }
  };
```

```
// データ取得
useEffect(() => {
  if (open && masterId) {
    fetchMasterData(masterId);
  }
}, [open, masterId]);

const fetchMasterData = async (id: string) => {
  setIsFetching(true);
  try {
    const response = await apiClient.get(
      `/api/v1/master/${masterType}/${id}`,
      {
        headers: {
          'x-organization-id': organizationId
        }
      }
    );

    if (response.data.success) {
      const data = response.data.data;
      const formValues: any = {
        isActive: data.isActive
      };

      // マスタ種別に応じてフィールド名を設定
      if (masterType === 'creator') {
        formValues.id = data.creatorId;
        formValues.name = data.creatorName;
      } else if (masterType === 'vendor') {
        formValues.id = data.vendorId;
        formValues.name = data.vendorName;
      } else if (masterType === 'instrument') {
        formValues.id = data.instrumentId;
        formValues.name = data.instrumentName;
        formValues.vendorId = data.vendorId;
      }

      form.setFieldsValue(formValues);
    }
  } catch (error: any) {
    const errorMessage =
      error.response?.data?.error?.message || 'マスタ情報の取得に失敗しました';
    message.error(errorMessage);
    onClose();
  } finally {
    setIsFetching(false);
  }
};
```

```
  const handleSubmit = async (values: MasterFormValues) => {
    if (!masterId) return;

    setIsLoading(true);
    try {
      const requestBody: any = {
        isActive: values.isActive
      };

      // マスタ種別に応じてフィールド名を設定
      if (masterType === 'creator') {
        requestBody.creatorName = values.name;
      } else if (masterType === 'vendor') {
        requestBody.vendorName = values.name;
      } else if (masterType === 'instrument') {
        requestBody.instrumentName = values.name;
        requestBody.vendorId = values.vendorId;
      }

      const response = await apiClient.put(
        `/api/v1/master/${masterType}/${masterId}`,
        requestBody,
        {
          headers: {
            'x-organization-id': organizationId
          }
        }
      );

      if (response.data.success) {
        message.success('マスタを更新しました');
        form.resetFields();
        onSuccess();
        onClose();
      }
    } catch (error: any) {
      const errorMessage =
        error.response?.data?.error?.message || 'マスタの更新に失敗しました';
      message.error(errorMessage);
    } finally {
      setIsLoading(false);
    }
  };

  const handleCancel = () => {
    form.resetFields();
    onClose();
  };

  return (
```

```
    <Modal
      title={labels.title}
      open={open}
      onCancel={handleCancel}
      footer={null}
      width={600}
      destroyOnClose
    >
      <Form
        form={form}
        layout="vertical"
        onFinish={handleSubmit}
      >
        {/* ID（読み取り専用） */}
        <Form.Item
          label={labels.id}
          name="id"
          extra="※IDは変更できません"
        >
          <Input disabled />
        </Form.Item>

        {/* 名前 */}
        <Form.Item
          label={labels.name}
          name="name"
          rules={[
            { required: true, message: '名前を入力してください' },
            { min: 1, max: 50, message: '名前は1〜50文字で入力してください' }
          ]}
          extra="最大50文字"
        >
          <Input placeholder="名前を入力" maxLength={50} />
        </Form.Item>

        {/* Vendor ID（Instrumentの場合のみ） */}
        {masterType === 'instrument' && (
          <Form.Item
            label="Vendor ID"
            name="vendorId"
            rules={[
              { required: true, message: 'Vendor IDを選択してください' }
            ]}
          >
            <Select
              placeholder="Vendor IDを選択"
              showSearch
              optionFilterProp="label"
              options={vendorOptions}
              style={{ width: '100%' }}
```

```
            />
          </Form.Item>
        )}

        {/* 状態 */}
        <Form.Item
          label="状態"
          name="isActive"
          rules={[{ required: true, message: '状態を選択してください' }]}
        >
          <Radio.Group>
            <Radio value={true}>アクティブ</Radio>
            <Radio value={false}>非アクティブ</Radio>
          </Radio.Group>
        </Form.Item>

        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading || isFetching}
              disabled={isLoading || isFetching}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

10. 変更検知と最適化

10.1. 変更なし判定

フォームの値が初期値と同じ場合、APIを呼び出さずに確認メッセージを表示:

```
const handleSubmit = async (values: MasterFormValues) => {
  // 初期値と比較
```

```
  const initialValues = form.getFieldsValue();
  const hasChanged =
    values.name !== initialValues.name ||
    values.isActive !== initialValues.isActive ||
    (masterType === 'instrument' && values.vendorId !== initialValues.vendorId);

  if (!hasChanged) {
    message.info('変更がありません');
    return;
  }

  // API呼び出し...
};
```

10.2. 確認ダイアログ

重要な変更（非アクティブ化）の場合、確認ダイアログを表示:

```
import { Modal } from 'antd';

const handleSubmit = async (values: MasterFormValues) => {
  // 非アクティブ化の場合は確認
  if (!values.isActive && initialValues.isActive) {
    Modal.confirm({
      title: 'マスタを非アクティブ化しますか？',
      content: '非アクティブにすると、このマスタを使用できなくなります。',
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: () => submitUpdate(values)
    });
  } else {
    submitUpdate(values);
  }
};
```

---

11. テスト要件

11.1. 単体テスト

テストケース

- データ取得機能のテスト
- バリデーション機能のテスト
- フォーム送信処理のテスト
- 変更検知機能のテスト
- Vendor一覧取得のテスト（Instrument）

11.2. 統合テスト

テストシナリオ

1. 正常なマスタ編集（Creator）
2. 正常なマスタ編集（Vendor）
3. 正常なマスタ編集（Instrument）
4. 名前のみ変更
5. 状態のみ変更
6. Vendor IDのみ変更（Instrument）
7. 変更なしで保存
8. 存在しないマスタの編集
9. 非アクティブ化の確認ダイアログ

---

12. アクセシビリティ

12.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

12.2. スクリーンリーダー対応

- 読み取り専用フィールドに `aria-readonly="true"` を設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

---

13. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-23	初版作成	Development Team